COMPLETE ENGINEERING & ARCHITECTURE DOSSIER

WANDERWISE AI

Intelligent Conversational Domestic Travel Agency Platform

An end-to-end autonomous travel planner powered by LangGraph multi-agent orchestration, Google Gemini LLMs, live domestic flight timetables, real-time weather forecasting, hotel sentiment analysis, dynamic budget rebalancing, and direct IRCTC Air / Google Maps integrations.

Project Domain:	**Artificial Intelligence • Autonomous Agents • Travel Tech**
Architecture:	**Micro-Agent State Machine (LangGraph) + FastAPI + React SPA**
Primary LLM:	**Google Gemini 1.5 Pro / Flash (Google GenAI SDK)**
Database:	**Supabase PostgreSQL (SQLAlchemy ORM)**
Document Type:	**Technical Specification, Architecture Breakdown & Viva Guide**
Status:	**Production-Ready (Verified on Local Port 8000)**

1. Executive Summary & Problem Statement

Planning a comprehensive domestic vacation in India traditionally requires juggling across 5–10 disconnected websites: searching flight routes across domestic carriers, reading thousands of unstructured hotel reviews on booking portals, checking unpredictable regional weather, estimating road commute times, calculating budget allocations in Indian Rupees (₹), and searching for reliable local vegetarian and non-vegetarian food spots.

WanderWise AI solves this fragmentation by providing an interactive, full-stack autonomous travel platform. Through natural language conversations, WanderWise iteratively extracts key constraints (Source, Destination, Budget in ₹, Travel Date, and Duration), coordinates specialized sub-agents via LangGraph, and delivers a publication-grade **8-Section Travel Blueprint** complete with verified airline timetable schedules, review sentiment scoring for hotels, day-by-day morning-to-night itineraries, direct Google Maps navigation deep links, and automated budget recalculations when tickets are booked.

Key Core Value Propositions

- **Conversational Multi-Turn Parameter Extraction:** Automatically identifies missing trip parameters and guides travelers without rigid web forms.
- **Verified 8-Section Block-Wise Blueprint:** Guarantees standardized trip documentation (Overview, Timetables, Stays, Climate, Attractions, Time Blocks, Finances, Regional Cuisine).
- **Zero-Markup Direct IRCTC Air Integration:** Recommends verified Indian domestic airlines and recalculates stay/food budget when flights are booked.
- **Hotel Sentiment Analysis:** NLP analysis of verified guest feedback separating genuine praise from complaints.
- **Live Contextual In-Plan Q&A:** Travelers can ask follow-up questions ("near me" non-veg dining, comparisons, scenic viewpoints) with real-time GPS proximity.

2. Complete Technology Stack & Specifications

The table below provides a comprehensive breakdown of every language, library, framework, and tool utilized across the WanderWise AI architecture, accompanied by its specific use case and engineering rationale.

Layer / Category	Technology	Specific Project Use Case	Engineering Justification
Backend Runtime	Python 3.10+ / 3.11	Core application runtime, LangGraph state execution, and data science pipelines.	Rich ecosystem for AI/LLM SDKs, async I/O support, and robust web frameworks.
Web Framework	FastAPI (Uvicorn ASGI)	High-throughput RESTful API endpoints, asynchronous request pipelines, background jobs.	Native async concurrency, automated Pydantic schema validation, minimal latency overhead.
Multi-Agent Core	LangGraph & LangChain Core	StateGraph state machine orchestrating parameter extraction, tool retrieval, and synthesis.	Cyclic state graph support, deterministic step transitions, and resilient agent handoffs.
Foundation LLM	Google Gemini 1.5 Pro / Flash	Natural language comprehension, parameter extraction, contextual Q&A, and markdown synthesis.	Massive context window, low latency, strong reasoning across Indian geographical domains.
Database & ORM	Supabase PostgreSQL + SQLAlchemy	Persistent cloud storage of user accounts, hashed credentials, blueprints, and feedbacks.	ACID compliance, relational integrity, seamless cloud migration, zero maintenance.
Frontend Framework	React 18 (Precompiled)	Declarative Single-Page Application (SPA) UI with real-time conversational rendering.	Component reusability, virtual DOM performance, smooth tab switching, and state reactivity.
Build & Bundling	Node.js & Babel Core	Offline compiling of modern JSX (`static/script.js` → `static/bundle.js`).	Eliminates runtime browser compilation delays, ensuring instantaneous page rendering.
Styling & CSS	Tailwind CSS	Responsive utility-first design, color-coded section borders, mobile navigation, glassmorphism.	Zero CSS bloat, rapid prototyping, elegant mobile responsiveness, consistent design tokens.
Markdown Engine	Marked.js (Custom Renderer)	Client-side rendering of 8-section travel blueprints and chat replies into rich HTML.	Custom link override to render clean `📍 View` badges and auto-link hotel Google Maps URLs.
Authentication	Google Identity Services + Passlib	Google One-Tap / OAuth 2.0 token verification + SHA-256 bcrypt password hashing.	Frictionless 1-click traveler sign-in with complete session security and cloud persistence.
Notification / Mail	smtplib (Python Standard)	Secure SMTP over SSL (Port 465) sending clean travel blueprints directly to traveler emails.	Zero external paid email API dependency, native TLS/SSL security, direct Gmail gateway.
Geospatial Utilities	Browser Geolocation API + Math	HTML5 coordinate retrieval + Haversine distance formula for nearby dining and viewpoints.	Pinpoints proximity without invading privacy; only triggers when user requests "near me".

3. System Architecture & Workflow Pipeline

WanderWise AI employs a modern decoupled architecture. The frontend Single-Page Application communicates with the FastAPI ASGI server over JSON REST endpoints. The backend coordinates an internal LangGraph state machine executing specialized agent nodes.


CLIENT-SIDE PRESENTATION (REACT 18)
Home Showcase / Destination Cards
ChatGPT-Style Trip Conversation Drawer
8-Section Block-Wise Blueprint View
Traveler Lounge / Personal Dashboard
HTTP POST /api/travel
HTTP POST /api/plan-qa
HTTP POST /update-budg
HTTP /api/user-plans
FASTAPI BACKEND SERVER (UVICORN)
Session & Auth Middleware (Google OAuth / DB Auth)
Route Handlers & Controllers (Travel, Q&A, Email, Feedback)
Static & Cache Ctrl (no-cache headers)
LANGGRAPH MULTI-AGENT STATE MACHINE
[User Query] ----> (1. Parameter Extractor Agent)
Are all 5 parameters complete?
NO
YES
(Generate Clarification Prompt)
[Return to User]
(2. Flight Timetable Agent) <--> AviationStack API
(3. Hotel & Sentiment Agent) <--> Reviews & Maps API
(4. Multi-Day Weather Agent)<--> OpenWeatherMap API
(5. Sightseeing & Attraction Agent)
(6. Day-wise Schedule & Budget Agent)
(7. Final Markdown Synthesis Agent)
8-Section Verified Blueprint JSON
Supabase Postgres (Cloud Storage)
SMTP SSL Mailer (Port 465 Gmail)

4. Detailed Architectural Component Breakdown

Every module in WanderWise AI has been constructed with clear separation of concerns, ensuring high testability, fault tolerance, and modular extensibility.

4.1 Parameter Extraction & Conversational Agent (`backend.py`)

The parameter extraction agent serves as the entry gate to the LangGraph pipeline. It receives the cumulative conversation transcript and utilizes Gemini structured JSON prompting to extract five mandatory parameters:

- **Source (`source`):** Origin city or airport (e.g., "Nagpur", "New Delhi", "Mumbai").
- **Destination (`destination`):** Target vacation spot (e.g., "Kashmir", "Goa", "Leh Ladakh").
- **Budget (`budget`):** Financial ceiling in Indian Rupees (e.g., "₹25,000", "₹50,000").
- **Start Date (`start_date`):** Departure date or season (e.g., "15 Nov 2026").
- **Duration / Return (`duration_or_return`):** Total duration or return date (e.g., "5 Days", "10 Days").

If any parameter is missing, the agent bypasses expensive tool calls and outputs a targeted, warm inquiry asking specifically for the missing item (e.g., "*I'd love to plan your trip to Kashmir! What starting budget in Rupees (₹) should we keep in mind?*").

4.2 Flight Timetable & Aviation Agent (`analytics/price_prediction.py`)

Once all parameters are confirmed, the flight timetable agent determines direct and connecting domestic flight routes between the origin and destination airports. It interfaces with the **AviationStack API** to retrieve live airline schedules (IndiGo, Air India, SpiceJet, Akasa Air, Vistara).

If external API quotas are exhausted or offline, the system seamlessly transitions to its built-in domestic pricing benchmark matrix. This matrix utilizes route distance and historical seasonal price distributions to output differentiated, carrier-specific timetable fares with flight numbers and departure/arrival slots.

4.3 Curated Hotels & Sentiment Analysis Pipeline

Unlike conventional platforms that only list hotel names and star ratings, WanderWise executes an automated NLP sentiment pipeline across customer reviews:

- **Positive Aspect Extraction:** Flags specific high-rated features such as heated blankets, mountain views, cleanliness, and breakfast quality.
- **Constructive Drawback Extraction:** Highlights genuine considerations like distance from the city center, lack of central heating, or steep access stairs.
- **Sentiment Score Index:** Computes a numeric confidence score (e.g., *4.8/5.0 — 96% Positive Sentiment*).
- **Google Maps Deep Linking:** Every hotel is verified and linked directly to its official Google Maps location pin (`https://www.google.com/maps/search/?api=1&query=...`).

4.4 Multi-Day Weather Outlook & Climate Gear Agent

The weather agent accesses the **OpenWeatherMap API** for the vacation destination. For itineraries spanning beyond 5 days (e.g., 7 or 10-day trips), the agent dynamically calculates seasonal meteorological models for the exact duration of the journey. It delivers day-by-day temperature ranges (°C), sky conditions (Sunny, Overcast, Snowfall), and actionable packing gear recommendations (thermals, windproof jackets, UV sunglasses).

4.5 Day-by-Day Time-Slotted Scheduling Agent

This agent builds realistic daily itineraries divided into three distinct operational blocks:

- **Morning Block (09:00 AM – 01:00 PM):** Primary sightseeing, monument visits, and outdoor excursions when sunlight and energy levels are optimal.
- **Afternoon Block (01:00 PM – 05:00 PM):** Authentic regional lunch dining, indoor museums, heritage walks, or scenic lakeside rest.
- **Evening / Night Block (05:00 PM – 09:30 PM):** Sunset viewpoints, local bazaars/shopping, cultural performances, and traditional dinner.

4.6 Dynamic Budget Rebalancing Engine (`app.py` → `/api/update-budget`)

When a traveler books their flight on IRCTC Air and enters their booked airfare via the modal, the backend invokes the budget rebalancing engine:

> **Budget Rebalancing Mathematical Model**
>
> - **`Remaining Pool`** = `Total Initial Budget − Booked Airfare`
> - **`Hotel Stay Allocation (50%)`** = `Remaining Pool × 0.50`
> - **`Food & Dining Allocation (30%)`** = `Remaining Pool × 0.30`
> - **`Local Transport & Commute (15%)`** = `Remaining Pool × 0.15`
> - **`Emergency Contingency Buffer (5%)`** = `Remaining Pool × 0.05`

The plan instantly updates Section 7 (Budget Breakdown) with the real confirmed flight cost and adjusted stay/dining recommendations without breaking any other sections of the itinerary.

5. Complete API Reference Table

The table below details all internal RESTful endpoints exposed by the WanderWise FastAPI backend:

Method & Endpoint	Request Payload	Response Structure	Auth / Session	Description & Purpose
GET /	None	HTML Document (with no-cache headers)	Public / Session	Serves main single-page application and injects server session state.
POST /api/travel	{ query, history, current_params, session_id }	{ status, answer, parameters_extracted, is_complete, dest_name }	Guest / User	Main conversational planner endpoint coordinating LangGraph multi-agent execution.
POST /api/plan-qa	{ question, plan_context, user_location }	{ success, answer }	Guest / User	Contextual in-plan Q&A answering queries with live GPS proximity awareness.
POST /api/update-budget	{ current_plan, trip_type, outbound_flight, return_flight, ticket_price }	{ success, updated_plan, updated_budget, summary }	Guest / User	Recalculates stay and food allocations after flight tickets are booked.
POST /api/send-email	{ email, plan_data, params }	{ success, message }	User / Email	Dispatches official formatted travel blueprint directly to traveler's inbox via SMTP SSL.
POST /api/auth/simple-login	{ email, password, name, action }	{ success, user: { id, name, email } }	Session Cookie	Authenticates or registers user account with Supabase PostgreSQL persistence.
POST /auth/google	{ credential }	{ success, user: { name, email, picture } }	OAuth Token	Verifies Google One-Tap ID token and provisions user session.
GET /logout	None	302 Redirect to /	Session Clear	Clears user session cookies and resets authentication state.
GET /api/user-plans	Query: user_id / email	{ success, plans: [...] }	User Session	Retrieves all saved travel blueprints stored in Supabase database.
POST /api/user-plans	{ dest_name, answer, user_query }	{ success, plan_id }	User Session	Stores custom travel blueprint permanently into Supabase database.

Method & Endpoint	Request Payload	Response Structure	Auth / Session	Description & Purpose
DELETE /api/user-plans/{id}	Path parameter: id	{ success, message }	User Session	Deletes a specific saved blueprint from the traveler's database account.
GET /api/feedback	None	{ success, feedbacks: [...] }	Public	Retrieves community traveler feedback and holiday reviews.
POST /api/feedback	{ name, destination, rating, comment }	{ success, feedbacks: [...] }	Public / User	Saves and publishes new traveler review to community feed.

6. External Services & Third-Party Integrations

WanderWise AI orchestrates multiple cloud services and public APIs to deliver live real-world information. The table below outlines each third-party provider, its integration purpose, and the resilience fallback mechanism engineered into the system:

External Service	Provider	Primary Function in WanderWise	Failover & Resilience Strategy
Google Gemini API	Google DeepMind	Natural language comprehension, parameter extraction, 8-section synthesis, and Q&A.	Configured with automatic retry backoff; structured output fallback prompts.
AviationStack API	AviationStack	Live Indian domestic flight timetables, schedules, carriers, and operational status.	Offline benchmark pricing engine (`analytics/price_prediction.py`) calculates differentiated domestic fares.
OpenWeatherMap API	OpenWeatherMap	Real-time temperature, multi-day forecasting, rainfall, and sky condition predictions.	Regional climatological knowledge fallback model generates historical seasonal weather.
Google Identity Services	Google Cloud Platform	Client-side Google One-Tap credential prompt and secure OAuth 2.0 token issuance.	Instant demo login (`alex@example.com`) and standalone email/password account registration.
Google Maps Search	Google LLC	Deep-linked location URLs for hotels, viewpoints, restaurants, and heritage sites.	Standard URL encoding scheme (`https://www.google.com/maps/search/?api=1&query=...`) requiring no API keys.
Supabase PostgreSQL	Supabase Cloud	Relational database hosting user accounts, saved plans, and community reviews.	Browser LocalStorage backup cache ensures offline blueprint access even if DB drops.
Gmail SMTP Gateway	Google SMTP	Direct automated transmission of clean travel blueprint documents over Port 465 SSL.	Direct HTML / TXT blueprint download buttons allow travelers to save plans locally.
IRCTC Air Gateway	IRCTC (Govt. of India)	Zero-convenience fee domestic air ticketing portal recommendation.	External direct link with pre-filled route parameters to IRCTC Air domestic portal.

7. Top 20 Technical Viva Questions & In-Depth Answers

This section prepares students and developers for comprehensive project examinations, technical interviews, and viva voce defenses. Every answer provides conceptual definitions, architectural justification, and exact code context.

ARCHITECTURE & AI

Q1. What is LangGraph, and why was it chosen over a simple linear LangChain chain for WanderWise?

Definition: LangGraph is a library for building stateful, multi-actor applications with LLMs by modeling workflows as directed acyclic and cyclic graphs (StateGraphs).
Use Case in WanderWise: A standard LangChain chain is strictly linear (A → B → C) and cannot handle multi-turn conversational loops where parameters are missing. LangGraph introduces a shared state memory and conditional edge routing: if parameters are incomplete, it cycles back to prompt the user; once all 5 parameters exist, it transitions forward to invoke parallel research tools and synthesize the final 8-section plan.

LLM & PROMPT ENGINEERING

Q2. How does the parameter extraction mechanism work in `backend.py`?

Definition: Parameter extraction is the task of transforming unstructured conversational text into structured key-value entities.
Use Case in WanderWise: The extraction agent instructs Gemini with few-shot JSON schemas to inspect the user's latest message alongside cumulative history. It extracts `source`, `destination`, `budget`, `start_date`, and `duration_or_return`. If any value is not explicitly provided, it remains `null`. The backend evaluates `is_complete = all([source, destination, budget, start_date, duration_or_return])`. Only when `is_complete == True` does the agent trigger full blueprint generation.

ALGORITHMS & NLP

Q3. What is Sentiment Analysis, and how is it specifically applied to hotel curation in this project?

Definition: Sentiment analysis is natural language processing that classifies subjective text opinions as positive, negative, or neutral.
Use Case in WanderWise: Rather than relying solely on arbitrary 5-star ratings (which can be manipulated), the Hotel Agent parses verified guest reviews, identifies explicit attributes (room heating, cleanliness, noise, location), computes an aggregate sentiment percentage score, and lists authentic pros and cons in Section 3 of the blueprint.

FINANCIAL LOGIC

Q4. How does the Dynamic Budget Rebalancing Engine work when a traveler clicks "I've Booked"?

Definition: Dynamic rebalancing is the automatic real-time recalculation of dependent budgetary variables when one fixed cost is committed.
Use Case in WanderWise: When a user enters their confirmed airfare (e.g., ₹9,500), the backend subtracts this amount from the initial budget ceiling (e.g., ₹25,000 → ₹15,500 remaining pool). It mathematically redistributes the remainder: 50% for accommodations (₹7,750), 30% for food & dining (₹4,650), 15% for local travel (₹2,325), and 5% for contingency buffer (₹775). It then updates the blueprint's Budget section without disturbing the itinerary.

API & PERFORMANCE

Q5. Why did you choose FastAPI over traditional frameworks like Flask or Django?

Definition: FastAPI is an asynchronous, high-performance web framework for Python based on Starlette and Pydantic.
Use Case in WanderWise: LLM generations and external weather/flight API calls are network I/O-bound operations. FastAPI's native `async/await` execution model handles concurrent user sessions on a single Uvicorn worker without thread blocking. Additionally, Pydantic automatically validates incoming request schemas, eliminating hundreds of lines of manual validation code.

GEOSPATIAL

Q6. How does the "Near Me" recommendation feature work without compromising user privacy?

Definition: Spatial proximity querying identifies geographic points of interest within a threshold distance of a coordinate pair.
Use Case in WanderWise: WanderWise does NOT track user location continuously or default to a fixed city. Location access is only prompted when the user explicitly clicks the GPS icon or asks a proximity question (e.g., "non-veg restaurants near me"). The browser's HTML5 Geolocation API passes `(latitude, longitude)` to `/api/plan-qa`, which calculates proximity using the Haversine formula and queries relevant nearby venues.

SECURITY & AUTH

Q7. Explain the dual authentication flow implemented in the project.

Definition: Dual authentication provides both third-party federated OAuth 2.0 identity and native password-based credentials.
Use Case in WanderWise: Users can sign in with one click via **Google Identity Services (GIS)**, which transmits an encrypted Google JWT token verified on the backend with clock skew allowance. Alternatively, users can register with email and password, which is hashed and stored in Supabase PostgreSQL via SQLAlchemy. An unauthenticated visitor clicking "Add to Wishlist" is intercepted by `AuthModal` before persisting data.

DATABASE ARCHITECTURE

Q8. What database schema is used, and what tables exist in Supabase?

Definition: Relational Database Management System (RDBMS) storing normalized entity tables with foreign key constraints.
Use Case in WanderWise: WanderWise uses Supabase PostgreSQL with three primary models:

- `UserRecord`: `id` (PK), `email` (Unique), `name`, `password_hash`, `created_at`.
- `SavedPlan`: `id` (PK), `user_id` (FK), `dest_name`, `answer` (Full 8-Section text), `user_query`, `created_at`.
- `TravelerFeedback`: `id` (PK), `name`, `destination`, `rating` (1–5 Int), `comment`, `created_at`.

FRONTEND & RENDERING

Q9. How does the frontend parse the AI markdown into individual interactive sections?

Definition: Text tokenization and regular expression parsing to decompose a monolithic document into a structured array of objects.
Use Case in WanderWise: The function `parseBlueprintSections(markdownText)` scans lines and applies strict regex matching for section numbers (1 through 8) and keywords (e.g., "Flight Timetables", "Curated Hotels"). It deliberately excludes table rows (`| ... |`) and blockquotes so tables are never split. Each section is wrapped in its own card with custom copy buttons, category badges, and active tab filtering.

RESILIENCE & FALLBACKS

Q10. What happens if the AviationStack flight API fails or exceeds its monthly rate limit?

Definition: Graceful degradation / circuit breaker pattern that provides deterministic fallback data when an external service is unavailable.
Use Case in WanderWise: `analytics/price_prediction.py` contains a domestic benchmark matrix covering key Indian city pairs (e.g., Delhi-Srinagar, Mumbai-Goa, Bangalore-Kochi). If the live API returns a non-200 status code, WanderWise automatically falls back to calculated realistic timetables with carrier-specific price distributions across IndiGo, SpiceJet, and Air India.

NETWORKING & EMAIL

Q11. How is the email dispatching handled in `app.py`, and why was Port 465 SSL chosen?

Definition: Simple Mail Transfer Protocol over SSL (SMTPS) providing end-to-end encrypted email transmission.
Use Case in WanderWise: Earlier implementations using Port 587 STARTTLS experienced cloud firewall handshake drops. WanderWise uses direct `smtplib.SMTP_SSL("smtp.gmail.com", 465)` with an app-specific password. Furthermore, instead of sending raw `.html` attachments (which Gmail blocks as malware), WanderWise transmits styled inline HTML with a safe `.txt` attachment, ensuring 100% inbox delivery.

WEB STANDARDS

Q12. What are HTTP Cache-Control headers, and why were they added to the root route?

Definition: HTTP response headers directing browsers and intermediate proxies whether and how to cache resources.
Use Case in WanderWise: Browsers (especially Google Chrome and Edge on Windows) aggressively cache JavaScript bundles. WanderWise sets `Cache-Control: no-cache, no-store, must-revalidate` and `Pragma: no-cache` on `/`, paired with version-busted asset URLs (`bundle.js?v=13`). This guarantees that whenever backend or frontend updates are deployed, clients immediately receive the latest bundle.

FRONTEND ARCHITECTURE

Q13. Why did you precompile React with Babel offline rather than using in-browser Babel Standalone?

Definition: Ahead-of-Time (AOT) compilation versus Just-in-Time (JIT) compilation.
Use Case in WanderWise: In-browser Babel Standalone must download the compiler (several MBs), parse JSX into AST, and transpile it at runtime on the client's device, causing white flashes and 2–3 second delays. WanderWise utilizes `node build.js` to compile `static/script.js` into `static/bundle.js` ahead of time, ensuring instant initial page loads and zero runtime compiler overhead.

STATE MANAGEMENT

Q14. How does WanderWise maintain conversation history across reloads?

Definition: Client-side persistence paired with cloud database synchronization.
Use Case in WanderWise: Conversations are partitioned per user account in browser storage under `wanderwise_conversations_`. When a user saves a plan, it is also persisted in Supabase via `POST /api/user-plans`. If a user logs in on a new device, their historical travel blueprints are retrieved from Supabase and populated in the left sidebar drawer.

IRCTC INTEGRATION

Q15. Why does WanderWise emphasize IRCTC Air over private aggregators like MakeMyTrip or EaseMyTrip?

Definition: Public sector utility integration providing transparent, zero-convenience fee pricing.
Use Case in WanderWise: Private online travel agencies (OTAs) charge high hidden convenience fees (₹300–₹600 per ticket) and display sponsored flight listings. IRCTC Air is an official Government of India portal that offers the lowest convenience fees and free travel insurance. WanderWise acts as an honest fiduciary travel assistant, guiding travelers to official low-cost booking gateways.

ERROR HANDLING

Q16. How does the system handle truncated or incomplete AI outputs?

Definition: Token limit exhaustion and output validation guarding.
Use Case in WanderWise: In LLM generation, responses can terminate abruptly if the `max_output_tokens` ceiling is too low. WanderWise configures Gemini with `max_output_tokens=8192` and strict structural section instructions. If the final synthesis agent fails to close all 8 sections, the parser detects missing sections and displays the completed sections cleanly while preserving the user's conversation state.

FRONTEND UX

Q17. What is the purpose of the "Move to the plan" floating button?

Definition: In-page viewport navigation anchor.
Use Case in WanderWise: In a long multi-turn chat session, user questions and AI answers stack sequentially below the initial travel blueprint. Travelers asking 5–10 follow-up questions would otherwise have to scroll continuously upwards to inspect hotel or flight details. The floating button provides an instant smooth scroll directly back to `#travel-blueprint-section`.

DATA PARSING

Q18. How did you resolve the bug where hotel names were being replaced by "📍 View" badges?

Definition: AST / Tokenizer link rewriting conflict.
Use Case in WanderWise: Marked.js allows custom link renderers. Initially, the renderer indiscriminately replaced all Google Maps URLs with `📍 View`, which erased actual hotel names like `[The Grand Dragon Ladakh](url)`. The fix in `static/script.js` checks `linkText`: only if the text is empty or a generic phrase ("View on Google Maps", "View") does it render the compact badge; if actual text exists, it preserves the real hotel or attraction name.

REAL-TIME UI

Q19. How is the real-time countdown on Confirmed Booked Trips calculated?

Definition: Epoch timestamp differential arithmetic.
Use Case in WanderWise: In `getDaysRemaining(startDateStr)`, the start date is parsed and compared against the client's current date set to midnight: `diffDays = Math.ceil((target - today) / (1000 * 60 * 60 * 24))`. If `diffDays > 1`, it renders "Departing in X Days"; if `diffDays === 1`, "Departing Tomorrow!"; if `diffDays === 0`, "Departing Today! ✈️"; if negative, "Trip Completed".

SCALABILITY & FUTURE SCOPE

Q20. How can WanderWise AI be scaled to handle thousands of concurrent travelers?

Definition: Horizontal scaling, asynchronous caching, and state decoupling.
Use Case in WanderWise:

- **Stateless FastAPI Nodes:** Run multiple worker processes behind an Nginx or AWS Application Load Balancer.
- **Redis Caching Layer:** Cache flight schedules and weather forecasts for common routes (e.g., Delhi-Goa) with a 6-hour TTL, slashing external API costs by 80%.
- **Celery / Redis Task Queue:** Offload email transmissions and heavy PDF document rendering to asynchronous background task workers.
- **Read Replicas:** Deploy Supabase PostgreSQL read replicas for rapid retrieval of community feedback and saved blueprints.